

Competitive Programming Bootcamp Day 0

Speaker Notes

Ryan Pelchat

August 2026

Session Goal

Day 0 should orient students to the ICPC-style contest environment before the bootcamp turns into algorithm practice. The main point is that competitive programming performance is a combination of technical skill, contest rules, team coordination, and disciplined execution under time pressure.

Slide 1: Title

- Welcome the class and frame Day 0 as orientation rather than a heavy algorithm lecture.
- Emphasize that the bootcamp will teach algorithms, but today answers: What are we training for? How does the contest work? How should a team operate?

Slide 2: Today's Roadmap

- Preview the four requested topics: what ICPC is, the rules, performance skills, and team building.
- Tell students that the rules discussion is practical, not legalistic: knowing the rules changes strategy and prevents avoidable mistakes.

Slide 3: What Is ICPC?

- Define ICPC as the International Collegiate Programming Contest.
- Explain that students work in teams and submit programs to an automatic judge.
- Stress that accepted code is the final product, not a written proof or partial explanation.
- Transition: the contest format shapes the way we practice.

Slide 4: What Makes ICPC Different?

- Walk through the diagram: three people, one computer, one clock, automatic judging, and visible contest pressure.
- Point out that a strong solo programmer can still struggle if the team cannot communicate or share the keyboard.
- Ask: What could go wrong if all three teammates try to solve the same problem silently?

Slide 5: The Contest Loop

- Describe the repeatable loop: read, model, solve, code, test, submit or debug.
- Explain that most contest mistakes happen when teams skip a step: coding before understanding constraints, submitting before testing, or debugging without a hypothesis.
- Tell students this loop will appear throughout the bootcamp.

Slide 6: Typical ICPC-Style Format

- Cover the common setup: three contestants, one workstation, several problems, around five hours, and judged submissions.
- Avoid overpromising exact official details. Mention that regions publish their own allowed languages, materials, and site rules.
- If students are preparing for a specific regional contest, assign them to read that regional rule page before the first practice contest.

Slide 7: Scoring and Ranking

- Explain the primary scoring rule: more solved problems beats fewer solved problems.
- Then explain penalty time: the accepted time for each solved problem plus penalties for wrong attempts before acceptance.
- Give the strategic implication: do not guess randomly, but do not be paralyzed by fear of wrong answers either.
- Mention that compile-error treatment and tie-breakers should be checked against local rules.

Slide 8: Scoring Example

- Compute the example slowly: Problem A contributes 18 minutes. Problem B contributes 73 minutes plus 40 minutes for two prior rejected attempts. Problem C contributes nothing because it is unsolved.
- Ask students which is better: solving one problem quickly or solving two problems with some penalty. The answer depends on solved count first.
- Reinforce that unsolved wrong attempts do not add penalty, but they still consume real time.

Slide 9: Contest Conduct

- Frame rules as preserving fairness: teams should solve with their own minds, approved tools, and official contest channels.
- Clarification requests go to judges through the contest system, not to friends, online forums, or instructors.
- Make the "when unsure" message explicit: ask a judge or proctor.
- For bootcamp practice, define which tools students may use during mock contests.

Slide 10: Skills Needed to Perform

- Explain that performance is broader than algorithm knowledge.
- Give a quick example for each row: constraints point toward complexity, testing catches off-by-one errors, strategy prevents wasting an hour on the wrong problem.
- Ask students to identify which skill area they currently feel strongest in and which one needs the most work.

Slide 11: Technical Skill Stack

- Break technical preparation into fluency, algorithms, and judgment.
- Fluency means students can use their language and standard library without searching for every detail.
- Algorithms are the reusable patterns the bootcamp will teach.
- Judgment is knowing which pattern applies, why it is correct, and whether it fits the constraints.

Slide 12: Team Skill Stack

- Define driver, navigator, and strategist as temporary roles.
- Driver is not "the best programmer"; driver is the person currently responsible for turning the plan into code.
- Navigator should actively read code and suggest tests, not passively watch.
- Strategist protects the team's time and helps decide whether to continue, switch, or ask for clarification.

Slide 13: First 15 Minutes of a Contest

- Explain that the opening minutes are about information gathering.
- Encourage teams to divide the problem set quickly, then regroup and compare difficulty estimates.
- Make the first target problem a team decision.
- Strong warning: do not let one teammate monopolize the computer before the team has an agreed plan.

Slide 14: Submission Discipline

- Present the checklist as a habit that saves time.
- Before coding: say algorithm and complexity out loud.
- Before submitting: run samples, tiny cases, and boundary cases.
- After rejection: avoid random edits. State a likely cause, test it, then change the code.

Slide 15: Icebreaker - Build the Team Map

- Put students in teams of three. If teams are not fixed yet, use temporary groups.
- Time box the activity: 8 minutes discussion, 1 minute share-out.
- Circulate and listen for whether students are naming concrete skills rather than vague labels.
- Share-out prompt: team name, one strength, one growth area, and their "We work best when..." sentence.

Slide 16: Team Challenge - One Keyboard Protocol

- Use the two-sum task because the algorithm is simple enough that the focus stays on communication.
- Enforce the rule that only the driver touches the keyboard.
- Watch whether the navigator asks useful questions and whether the strategist notices time.
- After 5 minutes, rotate roles even if the team is not finished.
- Debrief: What helped? What slowed the team down? Did everyone understand the code?

Slide 17: Team Contract

- Ask teams to write short answers, not essays.
- Encourage concrete phrases: "pause and prove it", "two more tests", "switch after 10 minutes with no progress".
- Tell teams the contract is a living document. They should update it after practice contests.

Slide 18: What Success Looks Like This Week

- Summarize expected Day 0 outcomes.
- Students should leave able to describe the contest, explain scoring, name key rules to verify, and use an initial team protocol.
- Connect to Day 1: now that students understand the environment, the next sessions build the problem-solving and algorithmic tools.

Slide 19: Sources Checked

- Mention that ICPC rules change by season and region. These notes use the 2026/27 regional and 2026 world finals pages as references.
- Recommended instructor action: before delivering this session for a real regional contest, open the local regional page and update allowed languages, materials, and site restrictions.

Slide 20: Closing

- Close with the central message: learn the rules, build the team, solve carefully.
- If time remains, let teams revise their contract after the one-keyboard challenge.

Instructor Checklist

- Print or share the official regional rules for the contest students are targeting.
- Prepare a simple two-sum prompt for the one-keyboard challenge.
- Decide whether bootcamp mock contests allow internet access, AI tools, editor autocomplete, and local notes.
- Collect team names and initial role assignments.
- Save team contracts for reference after the first practice contest.

Sources

- ICPC Regional Rules 2026/27: <https://icpc.global/regionals/rules>
- ICPC World Finals Rules 2026: <https://icpc.global/worldfinals/rules/>
- ICPC Regionals overview: <https://icpc.global/regionals/regionals>